Python Dictionaries

Dictionaryچیست؟

در پایتون، Dictionaryیا «دیکشنری» یکی از مهم‌ترین ساختارهای داده است که برای ذخیره اطلاعات به صورت:

key : value استفاده می‌شود

مثلاً فرض کنید اطلاعات یک خودرو را داریم:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
```

در این Dictionary سه اطلاعات ذخیره شده است:

Key	Value
brand	Ford
model	Mustang
year	1964

در اینجا:

"brand" → key

"Ford" → value

و:

"model" → key

"Mustang" → value

۲چرا به Dictionary نیاز داریم؟

فرض کنید بخواهیم اطلاعات یک دانشجو را ذخیره کنیم با متغیرهای معمولی می‌توانیم بنویسیم:

```
name = "Ali"
age = 22
major = "Computer Engineering"
```

```
gpa = 3.5
```

اما اگر بخواهیم اطلاعات را به عنوان یک مجموعه واحد نگهداری کنیم، Dictionaryبسیار مناسب‌تر است:

```
student = {
   "name": "Ali",
   "age": 22,
   "major": "Computer Engineering",
   "gpa": 3.5
}
```

اکنون تمام اطلاعات دانشجو داخل یک متغیر به نام studentقرار دارد

ساخت یکDictionary

Dictionaryبا علامت‌های {}ساخته می‌شود

مثال:

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz"
}
```

ساختار کلی:

```
dictionary_name = {
   key1: value1,
   key2: value2,
   key3: value3
}
```

مثلاً:

```
person = {
```

```
    "name": "John",
    "age": 30,
    "country": "Germany"
}
```

چاپDictionary

برای چاپ Dictionary از print()استفاده می‌کنیم:

```
student = {
    "name": "Ali",
    "age": 22,
    "city": "Shiraz"
}
print(student)
```

خروجی:

```
{'name': 'Ali', 'age': 22, 'city': 'Shiraz'}
```

Dictionaryاز Key و Value تشکیل شده است

مثلاً:

```
student = {
    "name": "Ali",
    "age": 22,
    "city": "Shiraz"
}
```

داریم:

```
name → Ali
age  → 22
city → Shiraz
```

بنابراین Keyشناسه یا نام اطلاعات است و Value خود اطلاعات است

۱- دسترسی به Value با استفاده از Key

برای دسترسی به یک مقدار، نام Dictionary و سپس Key را داخل []قرار می‌دهیم

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz"
}
print(student["name"])
```

خروجی:

```
Ali
```

مثال:

```
print(student["age"])
```

خروجی:

```
22
```

و:

```
print(student["city"])
```

خروجی:

```
Shiraz
```

نکته مهم :در Dictionary برخلاف List معمولاً با indexبه اطلاعات دسترسی پیدا نمی‌کنیم

List:

```
names = ["Ali", "Sara", "John"]
print(names[0])
```

Dictionary:

```
student = {
   "name": "Ali",
   "age": 22
}
print(student["name"])
```

در List از index استفاده می‌کنیم، ولی در Dictionary از key.

فرض کنید Dictionary زیر را داریم:

```
car = {
   "brand": "Ford",
   "model": "Mustang",
   "year": 1964
}
```

در Dictionary سه Key داریم:

```
brand
model
year
```

برای دریافت مقدار مربوط به یکKey ، نام Key را داخل []قرار می‌دهیم

مثلاً:

```
print(car["model"])
```

خروجی:

```
Mustang
```

یا:

```
print(car["brand"])
```

خروجی:

```
Ford
```

و:

```
print(car["year"])
```

خروجی:

```
1964
```

۲- تفاوت Dictionary باList

در List:

```
students = ["Ali", "Sara", "John"]
print(students[0])
```

خروجی:

```
Ali
```

در اینجا 0یک **index**است اما در Dictionary:

```
student = {
   "name": "Ali",
   "age": 22
}
print(student["name"])
```

در اینجا "name"یک **key**است بنابراین:

```
List       → Index
Dictionary → Key
```

این یکی از مهم‌ترین تفاوت‌های List و Dictionary است

اگر Key وجود نداشته باشد چه اتفاقی می‌افتد؟

فرض کنید:

```
student = {
   "name": "Ali",
```

```
    "age": 22
}
```

حالا بنویسیم:

```
print(student["city"])
```

چون Key به نام cityوجود ندارد، Pythonخطایی از نوع:

```
KeyError
```

ایجاد می‌کند

بنابراین قبل از استفاده از []باید بدانیم Key موردنظر وجود دارد

۳- استفاده از ()get

Pythonروش دیگری برای دریافت Value دارد:

```
student = {
    "name": "Ali",
    "age": 22
}
print(student.get("name"))
```

خروجی:

```
Ali
```

بنابراین:

```
student["name"]
```

و:

```
student.get("name")
```

در صورتی که Key وجود داشته باشد، نتیجه یکسانی دارند

تفاوت []و ()get

روش اول

```
print(student["city"])
```

اگر cityوجود نداشته باشد:

```
KeyError
```

روش دوم

```
print(student.get("city"))
```

اگر cityوجود نداشته باشد:

```
None
```

بنابراین:

روش	Keyوجود داشته باشد	Keyوجود نداشته باشد
dict[key]	Value	KeyError
dict.get(key)	Value	None

مقدار پیش‌فرض در ()get

یکی از قابلیت‌های بسیار مفید ()getاین است که می‌توانیم مقدار پیش‌فرض تعیین کنیم

```
student = {
   "name": "Ali",
   "age": 22
}
print(student.get("city", "Unknown"))
```

خروجی:

```
Unknown
```

یعنی:

اگر cityوجود داشت، مقدار آن را بده؛ در غیر این صورت "Unknown"را برگردان

مثال دیگر:

```
print(student.get("email", "Email not found"))
```

خروجی:

```
Email not found
```

۴- keys() چیست؟

متد keys()برای دریافت تمامKey های Dictionary استفاده می‌شود

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(car.keys())
```

خروجی:

```
dict_keys(['brand', 'model', 'year'])
```

نکته مهم:

keys()یک listمعمولی برنمی‌گرداند بلکه یک view objectاز نوع dict_keysبرمی‌گرداند

۵- Viewچیست؟

این مفهوم کمی مهم‌تر است وقتی می‌نویسیم: keys = car.keys() در واقع keysیک تصویر/نمای زنده ازKey های Dictionaryاست

مثلاً:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
keys = car.keys()
```

```
print(keys)
```

خروجی:

```
dict_keys(['brand', 'model', 'year'])
```

حالا Dictionary را تغییر می‌دهیم:

```
car["color"] = "white"
```

دوباره:

```
print(keys)
```

خروجی:

```
dict_keys(['brand', 'model', 'year', 'color'])
```

چرا ، چون keysیک View از Dictionary است و تغییرات Dictionary را مشاهده می‌کند

مثال :

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
keys = car.keys()
print("Before:")
print(keys)
car["color"] = "white"
print("After:")
print(keys)
```

خروجی:

```
Before:
dict_keys(['brand', 'model', 'year'])
```

After:

```
dict_keys(['brand', 'model', 'year', 'color'])
```

ما keysرا دوباره ایجاد نکردیم، اما تغییر Dictionary در keysدیده شد این همان مفهوم **View**است

۶- values() چیست؟

متد ()valuesتمامValue های Dictionary را در اختیار ما قرار می‌دهد

مثلاً:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(car.values())
```

خروجی:

```
dict_values(['Ford', 'Mustang', 1964])
```

باز هم توجه کنید که خروجی یک List معمولی نیست؛ بلکه یک View است

تغییر Value وView

مثال:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
values = car.values()
```

```
print(values)
car["year"] = 2020
print(values)
```

خروجی:

```
dict_values(['Ford', 'Mustang', 1964])
dict_values(['Ford', 'Mustang', 2020])
```

چون Value مربوط به yearتغییر کرده است، Viewنیز تغییر را نشان می‌دهد

۷-اضافه کردن یک Value جدید

مثلاً:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
values = car.values()
print(values)
car["color"] = "red"
print(values)
```

خروجی:

```
dict_values(['Ford', 'Mustang', 1964])
dict_values(['Ford', 'Mustang', 1964, 'red'])
```

۸- items() چیست؟

متد items()برای دریافت **Keyو Value با هم** استفاده می‌شود

```
car = {
  "brand": "Ford",
```

```
    "model": "Mustang",
    "year": 1964
}
print(car.items())
```

خروجی:

```
dict_items([
    ('brand', 'Ford'),
    ('model', 'Mustang'),
    ('year', 1964)
])
```

هر آیتم به صورت یک Tupleنمایش داده می‌شود:

```
('brand', 'Ford')
('model', 'Mustang')
('year', 1964)
```

یعنی:

```
(Key, Value)
```

۹-چرا items()مهم است؟

چون وقتی بخواهیم هم Key و هم Value را داشته باشیم، items()بسیار کاربردی است

مثلاً:

```
student = {
    "name": "Ali",
    "age": 22,
    "city": "Shiraz"
}
```

```
for key, value in student.items():
    print(key, value)
```

خروجی:

```
name Ali
age 22
city Shiraz
```

یا می‌توانیم زیباتر بنویسیم:

```
for key, value in student.items():
    print(f"{key}: {value}")
```

خروجی:

```
name: Ali
age: 22
city: Shiraz
```

۱۰- تفاوت keys()، values()و items()

Method	چه چیزی برمی‌گرداند؟
keys()	Keyها
values()	Valueها
items()	Key + Value

مثلاً:

```
student = {
   "name": "Ali",
   "age": 22
}
```

keys()

```
student.keys()
```

نتیجه:

```
name
age
```

values()

```
student.values()
```

نتیجه:

```
Ali
22
```

items()

```
student.items()
```

نتیجه:

```
name → Ali
age → 22
```

۱۱- items() و Tuple

وقتی می‌نویسیم: student.items() هر آیتم به شکل: ("name", "Ali") است

و این ساختار یک Tuple است: ("name", "Ali")

۱۲- بررسی وجود Key با in

برای اینکه بفهمیم یک Key در Dictionary وجود دارد یا خیر، از inاستفاده می‌کنیم

مثلاً:

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz"
}
if "name" in student:
   print("Name exists")
```

خروجی:

```
Name exists
```

اگر Key وجود نداشته باشد

```
if "email" in student:
   print("Email exists")
else:
   print("Email does not exist")
```

خروجی:

```
Email does not exist
```

نکته بسیار مهم درباره in

وقتی می‌نویسیم"name" in student

Pythonوجود **Key**را بررسی می‌کند، نه Value را

مثلاً:

```
student = {
   "name": "Ali",
   "age": 22
}
print("name" in student)
```

نتیجه:

```
True
```

اما:

```
print("Ali" in student)
```

نتیجه:

```
False
```

چون "Ali"یک Value است، نه Key.

بررسی وجودValue

اگر بخواهیم وجود Value را بررسی کنیم، از values()استفاده می‌کنیم:

```
student = {
   "name": "Ali",
   "age": 22
}
if "Ali" in student.values():
   print("Ali exists")
```

خروجی:

```
Ali exists
```

بنابراین:

```
"name" in student
```

برای **Key**

و:

```
"Ali" in student.values()
```

برای **Value**است

مثال کامل

مثال زیر تمام مطالب این درس را ترکیب می‌کند:

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz"
}
# Access
print(student["name"])
# get()
print(student.get("age"))
# keys()
print(student.keys())
# values()
print(student.values())
# items()
print(student.items())
# Check Key
if "city" in student:
   print("City exists")
```

مثال کاربردی: دفترچه تلفن

```
phone_book = {
   "Ali": "09120000000",
   "Sara": "09210000000",
   "John": "09350000000"
}
```

دریافت شماره:Ali

```
print(phone_book["Ali"])
```

یا:

```
print(phone_book.get("Ali"))
```

بررسی وجود شخص

```
if "Ali" in phone_book:
   print("Ali is in the phone book")
```

نمایش تمام افراد

```
for name in phone_book.keys():
   print(name)
```

نمایش تمام شماره‌ها

```
for phone in phone_book.values():
   print(phone)
```

نمایش نام و شماره

```
for name, phone in phone_book.items():
   print(f"{name}: {phone}")
```

مثال کاربردی: سیستم نمرات

Dictionaryبرای ذخیره نمرات بسیار مناسب است:

```
grades = {
   "Ali": 18,
   "Sara": 19,
   "John": 15,
   "Reza": 17
}
```

نمایش نمره:Ali

```
print(grades["Ali"])
```

نمایش تمام دانشجویان:

```
for name in grades.keys():
   print(name)
```

نمایش تمام نمرات:

```
for grade in grades.values():
   print(grade)
```

نمایش نام و نمره:

```
for name, grade in grades.items():
   print(f"{name}: {grade}")
```

۱۳- ترکیب items()با if

حالا یک مثال کاربردی‌تر:

```
grades = {
   "Ali": 18,
   "Sara": 19,
   "John": 9,
```

```
    "Reza": 17
}
for name, grade in grades.items():
    if grade >= 10:
        print(name, "Passed")
    else:
        print(name, "Failed")
```

خروجی:

```
Ali Passed
Sara Passed
John Failed
Reza Passed
```

یک نکته مهم درباره View

سه متد زیر:

keys()

values()

items()

Viewبرمی‌گردانند

یعنی:

```
keys = student.keys()
values = student.values()
items = student.items()
```

این View ها به Dictionary اصلی مرتبط هستند اگر Dictionary تغییر کند، Viewنیز تغییرات را نشان می‌دهد

اگر List واقعی بخواهیم چه؟

اگر واقعاً بخواهیم خروجی keys()را به یک List تبدیل کنیم، می‌توانیم از list()استفاده کنیم:

```
student = {
   "name": "Ali",
   "age": 22
}
keys = list(student.keys())
print(keys)
```

خروجی:

```
['name', 'age']
```

اکنون keysیک List واقعی است همین کار برای:Values

```
values = list(student.values())
```

و:Items

```
items = list(student.items())
```

Cheat Sheet

کد	عملیات
student["name"]	دریافت مقدار
student.get("name")	دریافت امن مقدار
student.get("email", "N/A")	مقدار پیش‌فرض
student.keys()	تمامKey ها
student.values()	تمامValue ها
student.items()	Key + Value
"name" in student	بررسیKey
"Ali" in student.values()	بررسیValue
list(student.keys())	تبدیل Keys بهList
list(student.values())	تبدیل Values بهList
list(student.items())	تبدیل Items بهList

تمرین ۱

Dictionaryزیر را در نظر بگیرید:

```
person = {
   "name": "Ali",
   "age": 25,
   "city": "Shiraz",
   "job": "Programmer"
}
```

دستورهای زیر را بنویسید:

۱. نام شخص را چاپ کنید
۲. سن را چاپ کنید
۳. شهر را چاپ کنید
۴. تمامKey ها را چاپ کنید
۵. تمامValue ها را چاپ کنید
۶. تمام Key وValue ها را چاپ کنید

تمرین ۲

بررسی کنید آیا Key زیر وجود دارد:

email

اگر وجود داشت بنویسید:

Email exists

در غیر این صورت:

Email does not exist

تمرین ۳

با ()getمقدار emailرا دریافت کنید اگر وجود نداشت، عبارت روبرو نمایش داده شود: Email not available

تمرین ۴

Dictionaryزیر را در نظر بگیرید:

```
grades = {
   "Ali": 18,
   "Sara": 19,
   "John": 8,
   "Reza": 15
}
```

با استفاده از items()فقط دانشجویانی را چاپ کنید که نمره آنها حداقل ۱۰ است

خروجی مورد انتظار:

```
Ali: 18
Sara: 19
Reza: 15
```

تمرین ۵

یک Dictionary به نام productsایجاد کنید:

```
products = {
   "Laptop": 1200,
   "Mouse": 30,
   "Keyboard": 50,
   "Monitor": 300
}
```

برنامه‌ای بنویسید که:

۱. تمام محصولات را نمایش دهد

۲. تمام قیمت‌ها را نمایش دهد

۳. نام و قیمت هر محصول را نمایش دهد

۴. بررسی کند آیا "Laptop"وجود دارد

۵. بررسی کند آیا قیمت 300در Dictionary وجود دارد

۶. محصولاتی که قیمت آنها بیشتر از 100است را نمایش دهد

تغییر Value با استفاده از Key

ساده‌ترین روش تغییر مقدار:

```
car["year"] = 2018
```

کد کامل:

```
car = {
   "brand": "Ford",
   "model": "Mustang",
   "year": 1964
}
car["year"] = 2018
print(car)
```

خروجی:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 2018}
```

در اینجا:

```
car["year"] = 2018
```

یعنی: مقدار مربوط به Key به نام yearرا به 2018تغییر بده

قبل و بعد از تغییر

برای درک بهتر:

قبل

year → 1964

دستور

```
car["year"] = 2018
```

بعد

year → 2018

بنابراین مقدار قبلی از بین می‌رود و مقدار جدید جایگزین آن می‌شود

تغییر چند مقدار

می‌توانیم چند مقدار را نیز تغییر دهیم:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
car["year"] = 2020
car["model"] = "Mustang GT"
print(car)
```

خروجی:

```
{
  'brand': 'Ford',
  'model': 'Mustang GT',
  'year': 2020
}
```

نکته بسیار مهم: اگر Key وجود نداشته باشد چه می‌شود؟

فرض کنید:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
```

حالا بنویسیم:

```
car["color"] = "red"
```

آیا خطا ایجاد می‌شود؟خیر چون اگر Key وجود نداشته باشد، Pythonآن را به Dictionary اضافه می‌کند

نتیجه:

```
{
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964,
  "color": "red"
}
```

بنابراین:

```
dictionary[key] = value
```

دو حالت دارد ، حالت اول Key وجود دارد Valueتغییر می‌کند حالت دوم Key وجود ندارد یک آیتم جدید اضافه می‌شود

مثال ساده برای تفاوت تغییر و اضافه کردن

```
student = {
  "name": "Ali",
  "age": 22
}
```

دستور:

```
student["age"] = 23
```

مقدار ageرا تغییر می‌دهد اما:

```
student["city"] = "Shiraz"
```

یک Key جدید ایجاد می‌کند

در نتیجه:

```
{
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
```

روش دوم()update :

روش دیگری برای تغییر Dictionary استفاده از متد:

()update است

مثلاً:

```
car = {
   "brand": "Ford",
   "model": "Mustang",
   "year": 1964
}
car.update({"year": 2020})
print(car)
```

خروجی:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 2020}
```

در اینجا:

```
car.update({"year": 2020})
```

مقدار yearرا تغییر می‌دهد

ساختار ()update

ساختار کلی:

```
dictionary.update({
   key: value
})
```

مثلاً:

```
student.update({
   "age": 25
})
```

تغییر چند مقدار با ()update

یکی از مزایای مهم ()updateاین است که می‌توانیم چند مقدار را همزمان تغییر دهیم

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Tehran"
}
student.update({
   "age": 23,
   "city": "Shiraz"
})
print(student)
```

خروجی:

```
{
   'name': 'Ali',
   'age': 23,
   'city': 'Shiraz'
}
```

یعنی دو مقدار با یک دستور تغییر کرده‌اند

()updateمی‌تواند آیتم جدید هم اضافه کند

این نکته بسیار مهم است اگر Key موردنظر وجود نداشته باشد، ()updateآن را اضافه می‌کند

مثلاً:

```
student = {
   "name": "Ali",
   "age": 22
}
student.update({
   "city": "Shiraz"
})
print(student)
```

خروجی:

```
{
   'name': 'Ali',
   'age': 22,
   'city': 'Shiraz'
}
```

بنابراین ()updateنیز مانند انتساب با []می‌تواند:

- مقدار موجود را تغییر دهد

- Keyجدید اضافه کند

مقایسه []و update()

دو روش زیر را مقایسه کنید

روش اول

```
student["age"] = 23
```

روش دوم

```
student.update({"age": 23})
```

هر دو می‌توانند مقدار ageرا تغییر دهند اما وقتی چند مقدار داریم، update()بسیار مناسب است:

```
student.update({
   "age": 23,
   "city": "Shiraz",
   "gpa": 3.5
})
```

مثال کاربردی: اطلاعات دانشجو

فرض کنید:

```
student = {
   "name": "Ali",
   "age": 22,
   "major": "Computer Engineering",
   "gpa": 3.2
}
```

می‌خواهیم معدل را تغییر دهیم:

```
student["gpa"] = 3.5
```

و:

```
print(student)
```

خروجی:

```
{
   'name': 'Ali',
   'age': 22,
   'major': 'Computer Engineering',
   'gpa': 3.5
}
```

تغییر چند اطلاعات دانشجو

```
student.update({
   "age": 23,
   "gpa": 3.7
})
```

نتیجه:

```
{
   "name": "Ali",
   "age": 23,
   "major": "Computer Engineering",
   "gpa": 3.7
}
```

استفاده از update()برای اضافه کردن اطلاعات

فرض کنید Dictionary اولیه:

```
student = {
   "name": "Ali",
   "age": 22
```

```
}
```

حالا:

```
student.update({
   "city": "Shiraz",
   "country": "Turkey"
})
```

Dictionaryتبدیل می‌شود به:

```
{
   "name": "Ali",
   "age": 22,
   "city": "Shiraz",
   "country": "Turkey"
}
```

ترکیب تغییر و اضافه کردن

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Tehran"
}
student.update({
   "age": 23,
   "city": "Shiraz",
   "gpa": 3.5
})
```

```
print(student)
```

در اینجا age تغییر کرده استcity تغییر کرده است و gpa اضافه شده است

نتیجه:

```
{
   'name': 'Ali',
   'age': 23,
   'city': 'Shiraz',
   'gpa': 3.5
}
```

یک مثال با محصول

فرض کنید فروشگاه داریم:

```
product = {
   “name”: “Laptop”,
   “price”: 1000,
   “stock”: 10
}
```

قیمت را تغییر می‌دهیم:

```
product[“price”] = 900
```

موجودی را تغییر می‌دهیم:

```
product[“stock”] = 15
```

دسته‌بندی را اضافه می‌کنیم:

```
product[“category”] = “Computer”
```

نتیجه:

```
{
   "name": "Laptop",
   "price": 900,
   "stock": 15,
   "category": "Computer"
}
```

همین کار با update()

می‌توانیم تمام تغییرات بالا را با یک دستور انجام دهیم:

```
product.update({
   "price": 900,
   "stock": 15,
   "category": "Computer"
})
```

این یکی از مهم‌ترین کاربردهای update()است

update()و Dictionary دیگر

می‌توانیم یک Dictionary را با Dictionary دیگری به‌روزرسانی کنیم

مثلاً:

```
student = {
   "name": "Ali",
   "age": 22
}
new_data = {
   "age": 23,
   "city": "Shiraz"
```

```
}
student.update(new_data)
print(student)
```

خروجی:

```
{
   'name': 'Ali',
   'age': 23,
   'city': 'Shiraz'
}
```

در اینجا age قبلاً وجود داشت، بنابراین تغییر کرد اما city وجود نداشت، بنابراین اضافه شد

یک مثال مهم با ()input

می‌توانیم اطلاعات دانشجو را از کاربر دریافت کنیم:

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Tehran"
}
city = input("Enter new city: ")
student["city"] = city
print(student)
```

اگر کاربر وارد کند:

```
Shiraz
```

نتیجه:

```
{
   'name': 'Ali',
```

```
    'age': 22,
    'city': 'Shiraz'
}
```

مثال با update() وInput

```
student = {
    "name": "Ali",
    "age": 22
}
age = int(input("Enter new age: "))
city = input("Enter city: ")
student.update({
    "age": age,
    "city": city
})
print(student)
```

مثلاً:

```
Enter new age: 23
Enter city: Shiraz
```

نتیجه:

```
{
    'name': 'Ali',
    'age': 23,
    'city': 'Shiraz'
}
```

یک نکته مهم درباره ()update

آرگومان ()updateباید اطلاعاتی شامل **Key-Value pairs**داشته باشد

رایج‌ترین حالت:

```
student.update({
   "age": 23,
   "city": "Shiraz"
})
```

یعنی یک Dictionary به ()updateداده‌ایم

اشتباه رایج

دانشجوی مبتدی ممکن است بنویسد:

```
student.update("age", 23)
```

این روش صحیح نیست ، روش صحیح:

```
student.update({
   "age": 23
})
```

یا برای چند مقدار:

```
student.update({
   "age": 23,
   "city": "Shiraz"
})
```

نکته مهم‌تر ()update :مقدار برنمی‌گرداند

مثلاً:

```
student = {
   "name": "Ali",
```

```
    "age": 22
}
result = student.update({
    "age": 23
})
print(result)
```

خروجی:

```
None
```

چرا؟ چون update()خود Dictionary را تغییر می‌دهد و Dictionary جدیدی را به عنوان نتیجه برنمی‌گرداند

پس این کار اشتباه است:

```
student = student.update({"age": 23})
```

چون در این صورت studentتبدیل به Noneمی‌شود

روش درست:

```
student.update({"age": 23})
```

ترکیب update()با get()

می‌توانیم متدهای Dictionary را با هم ترکیب کنیم

مثلاً:

```
student = {
    "name": "Ali",
    "age": 22
}
age = student.get("age")
print(age)
```

و سپس:

```
student.update({
    "age": age + 1
})
```

نتیجه:

```
{
    "name": "Ali",
    "age": 23
}
```

ترکیب Dictionary با if

مثلاً:

```
student = {
    "name": "Ali",
    "age": 22,
    "gpa": 2.8
}
if student["gpa"] < 3:
    student["status"] = "Needs Improvement"
print(student)
```

نتیجه:

```
{
    'name': 'Ali',
    'age': 22,
    'gpa': 2.8,
    'status': 'Needs Improvement'
```

```
}
```

مثال کامل‌تر: سیستم نمرات

```
student = {
    "name": "Ali",
    "math": 18,
    "python": 16,
    "database": 15
}
```

فرض کنید نمره Python اصلاح شده است:

```
student["python"] = 19
```

حالا میانگین را محاسبه کنیم:

```
average = (
    student["math"] +
    student["python"] +
    student["database"]
) / 3
student["average"] = average
print(student)
```

نتیجه شامل average نیز خواهد شد

یک پروژه کوچک

سیستم مدیریت اطلاعات دانشجو

Dictionaryاولیه:

```
student = {
    "name": "Ali",
    "age": 22,
```

```
    "major": "Computer Engineering",
    "gpa": 3.2
}
```

مرحله اول: تغییر معدل

```
student["gpa"] = 3.6
```

مرحله دوم: تغییر سن

```
student["age"] = 23
```

مرحله سوم: اضافه کردن شهر

```
student["city"] = "Shiraz"
```

مرحله چهارم: نمایش اطلاعات

```
for key, value in student.items():
    print(f"{key}: {value}")
```

خروجی:

```
name: Ali
age: 23
major: Computer Engineering
gpa: 3.6
city: Shiraz
```

مثال حرفه‌ای‌تر با ()update

همان پروژه را می‌توانیم بهتر بنویسیم:

```
student = {
    "name": "Ali",
    "age": 22,
    "major": "Computer Engineering",
    "gpa": 3.2
```

```
}
student.update({
    "age": 23,
    "gpa": 3.6,
    "city": "Shiraz"
})
for key, value in student.items():
    print(f"{key}: {value}")
```

تمرین ۱

Dictionaryزیر را ایجاد کنید:

```
car = {
    "brand": "BMW",
    "model": "X5",
    "year": 2020
}
```

سال خودرو را به 2025تغییر دهید

تمرین ۲

Dictionaryزیر را تغییر دهید:

```
student = {
    "name": "Ali",
    "age": 20,
    "city": "Tehran"
}
```

سن را به 21و شهر را به Shirazتغییر دهید

تمرین ۳

به Dictionary زیر:

```
product = {
   "name": "Laptop",
   "price": 1000
}
```

با استفاده از update()موارد زیر را اضافه کنید:

brand

stock

category

تمرین ۴

Dictionaryزیر را در نظر بگیرید:

```
student = {
   "name": "Ali",
   "age": 22,
   "gpa": 2.8
}
```

اگر معدل کمتر از 3بود، مقدار زیر را به Dictionary اضافه کنید:

status = "Needs Improvement"

تمرین ۵

یک Dictionary برای یک محصول ایجاد کنید:

```
product = {
   "name": "Laptop",
```

```
    "price": 1200,
    "stock": 10
}
```

برنامه‌ای بنویسید که:

۱. قیمت جدید را از کاربر دریافت کند
۲. موجودی جدید را از کاربر دریافت کند
۳. هر دو مقدار را با update() تغییر دهد
۴. یک category جدید به محصول اضافه کند
۵. در پایان تمام اطلاعات محصول را با items() چاپ کند

Add Dictionary Items

۱- اضافه کردن یک آیتم جدید

برای اضافه کردن یک آیتم جدید، کافی است یک **Key جدید** ایجاد کرده و مقدار آن را مشخص کنیم

مثلاً:

```
car["color"] = "red"
```

کد کامل:

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
car["color"] = "red"
print(car)
```

خروجی:

```
{
```

```
  'brand': 'Ford',
  'model': 'Mustang',
  'year': 1964,
  'color': 'red'
}
```

۲- مفهوم dictionary[key] = value

این ساختار را دانشجو باید کاملاً یاد بگیرد:

```
dictionary[key] = value
```

مثلاً:

```
student["name"] = "Ali"
```

یعنی:

```
Key   = name
Value = Ali
```

یا:

```
student["age"] = 22
```

یعنی:

```
Key   = age
Value = 22
```

۳- اضافه کردن چند آیتم

می‌توانیم چند Key جدید اضافه کنیم:

```
student = {
  "name": "Ali",
  "age": 22
}
```

```
student["city"] = "Shiraz"
student["country"] = "Turkey"
student["major"] = "Computer Engineering"
print(student)
```

نتیجه:

```
{
   'name': 'Ali',
   'age': 22,
   'city': 'Shiraz',
   'country': iran,
   'major': 'Computer Engineering'
}
```

۶- نکته بسیار مهم Add :یاChange ؟

فرض کنید:

```
student = {
   "name": "Ali",
   "age": 22
}
```

اگر بنویسیم:

```
student["age"] = 23
```

چون ageقبلاً وجود دارد، مقدار آن تغییر می‌کند

اما اگر بنویسیم:

```
student["city"] = "Shiraz"
```

چون cityوجود ندارد، یک آیتم جدید اضافه می‌شود

۷- روش دوم: استفاده از ()update

روش دیگری برای اضافه کردن آیتم استفاده از:

update() است

مثلاً:

```
car = {
   "brand": "Ford",
   "model": "Mustang",
   "year": 1964
}
car.update({
   "color": "red"
})
print(car)
```

خروجی:

```
{
   'brand': 'Ford',
   'model': 'Mustang',
   'year': 1964,
   'color': 'red'
}
```

۸- ساختار update()

ساختار کلی:

```
dictionary.update({
   "key": "value"
})
```

مثلاً:

```
student.update({
   "city": "Shiraz"
})
```

۹-اضافه کردن چند آیتم با ()update

یکی از مزیت‌های ()updateاین است که می‌توانیم چند آیتم را همزمان اضافه کنیم

```
student = {
   "name": "Ali",
   "age": 22
}
student.update({
   "city": "Shiraz",
   "country": "Turkey",
   "major": "Computer Engineering"
})
print(student)
```

نتیجه:

```
{
   'name': 'Ali',
   'age': 22,
   'city': 'Shiraz',
   'country': iran,
   'major': 'Computer Engineering'
}
```

۱۰- مقایسه دو روش

روش اول

```
student["city"] = "Shiraz"
student["country"] = "Turkey"
```

روش دوم

```
student.update({
   "city": "Shiraz",
   "country": "Turkey"
})
```

هر دو روش صحیح هستند اما برای اضافه یا تغییر چند آیتم، update()خواناتر است

۱۱- update() فقط برای Add نیست

نکته مهم:

update()هم برای اضافه کردن و هم برای تغییر دادن استفاده می‌شود

مثلاً:

```
student = {
   "name": "Ali",
   "age": 22
}
```

اگر بنویسیم:

```
student.update({
   "age": 23
})
```

مقدار ageتغییر می‌کند اما:

```
student.update({
```

```
    "city": "Shiraz"
})
```

یک آیتم جدید اضافه می‌کند

۱۲- مثال کاربردی: اطلاعات دانشجو

فرض کنید ابتدا فقط اطلاعات پایه را داریم:

```
student = {
    "name": "Ali",
    "age": 22
}
```

حالا اطلاعات بیشتری اضافه می‌کنیم:

```
student["city"] = "Shiraz"
student["major"] = "Computer Engineering"
student["gpa"] = 3.5
```

در پایان:

```
print(student)
```

خروجی:

```
{
    'name': 'Ali',
    'age': 22,
    'city': 'Shiraz',
    'major': 'Computer Engineering',
    'gpa': 3.5
}
```

۱۳-اضافه کردن اطلاعات با input()

حالا برنامه را کمی واقعی‌تر کنیم.

```
student = {
   "name": "Ali",
   "age": 22
}
city = input("Enter your city: ")
student["city"] = city
print(student)
```

اگر کاربر وارد کند:

```
Enter your city: Shiraz
```

Dictionaryبه شکل زیر می‌شود:

```
{
   'name': 'Ali',
   'age': 22,
   'city': 'Shiraz'
}
```

۱۴- اضافه کردن چند اطلاعات از کاربر

```
student = {}
name = input("Enter name: ")
age = int(input("Enter age: "))
city = input("Enter city: ")
student["name"] = name
student["age"] = age
```

```
student["city"] = city
print(student)
```

مثلاً کاربر:

```
Enter name: Ali
Enter age: 23
Enter city: Shiraz
```

نتیجه:

```
{
   'name': 'Ali',
   'age': 23,
   'city': 'Shiraz'
}
```

۱۵- همان برنامه با ()update

می‌توانیم همان برنامه را با ()updateبنویسیم:

```
student = {}
name = input("Enter name: ")
age = int(input("Enter age: "))
city = input("Enter city: ")
student.update({
   "name": name,
   "age": age,
   "city": city
})
print(student)
```

۱۶- مثال کاربردی: فروشگاه

فرض کنید اطلاعات یک محصول را داریم:

```
product = {
   "name": "Laptop",
   "price": 1200
}
```

حالا می‌خواهیم موجودی را اضافه کنیم:

```
product["stock"] = 10
```

دسته‌بندی را اضافه کنیم:

```
product["category"] = "Computer"
```

و برند را اضافه کنیم:

```
product["brand"] = "Dell"
```

در پایان:

```
print(product)
```

نتیجه:

```
{
   'name': 'Laptop',
   'price': 1200,
   'stock': 10,
   'category': 'Computer',
   'brand': 'Dell'
}
```

۱۷- اضافه کردن چند اطلاعات با ()update

روش کوتاه‌تر:

```
product.update({
   "stock": 10,
   "category": "Computer",
   "brand": "Dell"
})
```

۱۸- Dictionary خالی

یکی از کاربردهای مهم Dictionary این است که ابتدا یک Dictionary خالی ایجاد کنیم:

```
student = {}
```

سپس اطلاعات را به تدریج به آن اضافه کنیم:

```
student["name"] = "Ali"
student["age"] = 22
student["city"] = "Shiraz"
```

در پایان:

```
print(student)
```

نتیجه:

```
{
   'name': 'Ali',
   'age': 22,
   'city': 'Shiraz'
}
```

این روش در برنامه‌های واقعی بسیار رایج است

۱۹- استفاده از if هنگام اضافه کردن اطلاعات

فرض کنید می‌خواهیم فقط در صورتی status را اضافه کنیم که معدل دانشجو حداقل ۱۰ باشد:

```
student = {
   "name": "Ali",
   "gpa": 15
}
if student["gpa"] >= 10:
   student["status"] = "Passed"
print(student)
```

خروجی:

```
{
   'name': 'Ali',
   'gpa': 15,
   'status': 'Passed'
}
```

اگر معدل کمتر از ۱۰ باشد:

```
student = {
   "name": "John",
   "gpa": 8
}
if student["gpa"] >= 10:
   student["status"] = "Passed"
print(student)
```

status اضافه نمی‌شود

۲۰-استفاده از if...else

می‌توانیم برنامه را کامل‌تر کنیم:

```
student = {
   "name": "John",
   "gpa": 8
}
if student["gpa"] >= 10:
   student["status"] = "Passed"
else:
   student["status"] = "Failed"
print(student)
```

خروجی:

```
{
   'name': 'John',
   'gpa': 8,
   'status': 'Failed'
}
```

این مثال برای دانشجویان بسیار مهم است، چون نشان می‌دهد Dictionary می‌تواند به عنوان ساختار داده اصلی یک برنامه مورد استفاده قرار گیرد

۲۱-استفاده از for برای نمایش اطلاعات

بعد از اینکه آیتم‌ها را اضافه کردیم، می‌توانیم با items() آنها را نمایش دهیم:

```
student = {
   "name": "Ali",
   "age": 22
}
```

```
student["city"] = "Shiraz"
student["major"] = "Computer Engineering"
for key, value in student.items():
    print(f"{key}: {value}")
```

خروجی:

```
name: Ali
age: 22
city: Shiraz
major: Computer Engineering
```

۲۲-یک مثال کامل‌تر

در این مثال تمام مباحث قبلی را ترکیب می‌کنیم:

```
student = {}
student["name"] = "Ali"
student["age"] = 22
student.update({
    "city": "Shiraz",
    "major": "Computer Engineering",
    "gpa": 3.5
})
if student["gpa"] >= 3:
    student["status"] = "Good Student"
for key, value in student.items():
    print(f"{key}: {value}")
```

خروجی:

```
name: Ali
age: 22
city: Shiraz
major: Computer Engineering
gpa: 3.5
status: Good Student
```

۲۳- تفاوت Dictionary با List در Add

در List معمولاً از:

```
list.append()
```

استفاده می‌کنیم:

```
students = ["Ali", "Sara"]
students.append("John")
```

اما در Dictionary:

```
student = {
   "name": "Ali"
}
student["age"] = 22
```

۲۴مثال: سیستم ثبت اطلاعات دانشجو

یک تمرین بسیار خوب برای کلاس:

```
student = {}
student["name"] = input("Enter name: ")
student["age"] = int(input("Enter age: "))
student["major"] = input("Enter major: ")
```

```
print("\nStudent Information:")
for key, value in student.items():
    print(f"{key}: {value}")
```

مثلاً:

```
Enter name: Ali
Enter age: 23
Enter major: Computer Engineering
Student Information:
name: Ali
age: 23
major: Computer Engineering
```

۲۵- مثال پروژه‌ای: ثبت محصول

```
product = {}
product["name"] = input("Enter product name: ")
product["price"] = float(input("Enter price: "))
product["stock"] = int(input("Enter stock: "))
print("\nProduct Information:")
for key, value in product.items():
    print(f"{key}: {value}")
```

این مثال می‌تواند مقدمه‌ای برای پروژه‌های بزرگ‌تر فروشگاهی باشد

Cheat Sheet-۲۸

Syntax	کار
d = {}	ایجاد Dictionary خالی
d["key"] = value	اضافه کردن یک آیتم
d.update({...})	اضافه کردن چند آیتم
d["key"] = new_value	تغییر مقدار
d.update({"key": value})	اضافه کردن باupdate
print(d)	نمایشDictionary
d.keys()	نمایشKey ها
d.values()	نمایشValue ها
d.items()	نمایش Key وValue

تمرین ۱

Dictionaryزیر را ایجاد کنید:

```
car = {
   "brand": "BMW",
   "model": "X5"
}
```

موارد زیر را به آن اضافه کنید:

```
year = 2025
color = "Black"
price = 50000
```

تمرین ۲

یک Dictionary خالی به نام studentایجاد کنید و موارد زیر را با استفاده از []اضافه کنید:

name

age

city

major

تمرین ۳

تمرین قبل را این بار با update()انجام دهید

تمرین ۴

Dictionaryزیر را در نظر بگیرید:

```
product = {
   "name": "Laptop",
   "price": 1000
}
```

با استفاده از update()موارد زیر را اضافه کنید:

```
brand = "Lenovo"
stock = 20
category = "Computer"
```

تمرین ۵

یک Dictionary خالی به نام studentایجاد کنید

اطلاعات زیر را از کاربر دریافت کنید:

name

age

city

major

gpa

و آنها را داخل Dictionary ذخیره کنید

در پایان اطلاعات را با استفاده از:

```
for key, value in student.items():
```

نمایش دهید

تمرین ۶

یک Dictionary برای یک محصول ایجاد کنید:

```
product = {}
```

از کاربر دریافت کنید:

- نام محصول
- قیمت
- تعداد موجودی
- برند
- دسته‌بندی

سپس:

۱. اطلاعات را به Dictionary اضافه کنید

۲. اگر قیمت بیشتر از 1000بود، Keyزیر را اضافه کنید :

```
expensive = True
```

۳. در غیر این صورت :

```
expensive = False
```

۴. در پایان تمام اطلاعات محصول را نمایش دهید

Remove Dictionary Items

۱- روش‌های حذف ازDictionary

چهار روش اصلی:

1. pop()
2. popitem()
3. del

4. clear()

اما کاربرد آنها متفاوت است

روش	کاربرد
pop()	حذف یک Key مشخص
popitem()	حذف آخرین آیتم
del	حذف یک آیتم یا کلDictionary
clear()	حذف تمام آیتم‌ها و خالی کردنDictionary

۲-روش اول()pop :

متد ()popبرای حذف یک آیتم بر اساس Keyاستفاده می‌شود

ساختار:

```
dictionary.pop(key)
```

مثلاً:

```
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
car.pop("model")
print(car)
```

خروجی:

```
{'brand': 'Ford', 'year': 1964}
```

یعنی:

```
car.pop("model")
```

باعث شد:

model → Mustang

حذف شود

۳- مفهوم ()pop

اگر می‌دانیم دقیقاً کدام Key را می‌خواهیم حذف کنیم، ()popیکی از روش‌های مناسب است

مثلاً:

```
student.pop("age")
```

یعنی:

آیتم مربوط به Key به نام ageرا حذف کن

۴- ()pop مقدار حذف‌شده را برمی‌گرداند

یکی از نکات جالب ()popاین است که مقدار آیتم حذف‌شده را برمی‌گرداند.

مثلاً:

```
car = {
   "brand": "Ford",
   "model": "Mustang",
   "year": 1964
}
removed_value = car.pop("model")
print(removed_value)
print(car)
```

خروجی:

```
Mustang
{'brand': 'Ford', 'year': 1964}
```

بنابراین:

```
removed_value = car.pop("model")
```

دو کار انجام می‌دهد:

۱. آیتم modelرا حذف می‌کند

۲. مقدار آن (Mustang) را برمی‌گرداند

۷- pop() اگر Key وجود نداشته باشد

فرض کنید:

```
student = {
   "name": "Ali",
   "age": 22
}
```

و بنویسیم:

```
student.pop("city")
```

چون cityوجود ندارد، Pythonخطا می‌دهد:

```
KeyError
```

بنابراین اگر مطمئن نیستیم Key وجود دارد، باید مراقب باشیم

۸- استفاده از مقدار پیش‌فرض در pop()

یکی از ویژگی‌های مفید pop()این است که می‌توانیم مقدار پیش‌فرض مشخص کنیم

```
student = {
   "name": "Ali",
   "age": 22
}
result = student.pop("city", "Not Found")
print(result)
```

خروجی:

```
Not Found
```

در این حالت برنامه خطا نمی‌دهد

ساختار:

```
dictionary.pop(key, default_value)
```

مثلاً:

```
student.pop("phone", "Phone does not exist")
```

۹-روش دوم popitem() :

روش دوم:

```
popitem()
```

این متد آخرین آیتم واردشده به Dictionary را حذف می‌کند

مثلاً:

```
car = {
   "brand": "Ford",
   "model": "Mustang",
   "year": 1964
}
car.popitem()
print(car)
```

خروجی:

```
{'brand': 'Ford', 'model': 'Mustang'}
```

چون:

year → 1964 آخرین آیتم بود، حذف شد

۱۰-چرا popitem() آخرین آیتم را حذف می‌کند؟

از Python 3.7 به بعدDictionary ها ترتیب ورود آیتم‌ها را حفظ می‌کنند

مثلاً:

```
student = {}
```

```
student["name"] = "Ali"
student["age"] = 22
student["city"] = "Shiraz"
```

ترتیب ورود:

1 → name

2 → age

3 → city

بنابراین:

```
student.popitem()
```

cityرا حذف می‌کند

۱۱- popitem() چه چیزی برمی‌گرداند؟

popitem()یک tupleشامل Key و Value حذف‌شده را برمی‌گرداند

مثلاً:

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz"
}
item = student.popitem()
print(item)
```

خروجی:

```
('city', 'Shiraz')
```

بنابراین می‌توانیم بنویسیم:

```
key, value = student.popitem()
```

```
print(key)
print(value)
```

خروجی:

```
city
Shiraz
```

۱۲-تفاوت ()popو ()popitem

()pop: می‌گوییم **کدام Key** حذف شود

```
student.pop("age")
```

popitem()

نمی‌گوییم کدام.Key

Pythonآخرین آیتم را حذف می‌کند:

```
student.popitem()
```

مقایسه:

popitem()	pop()	ویژگی
خیر	بله	انتخابKey
خیر	بله	حذف Key مشخص
بله	خیر	حذف آخرین آیتم
بله	بله	مقدار حذف‌شده را برمی‌گرداند
خیر	بله	امکانdefault value

۱۳- روش سومdel :

روش دیگری برای حذف آیتم del است

مثلاً:

```
car = {
  "brand": "Ford",
```

```
  "model": "Mustang",
  "year": 1964
}
del car["model"]
print(car)
```

خروجی:

```
{'brand': 'Ford', 'year': 1964}
```

۱۴- del چگونه کار می‌کند؟

ساختار:

```
del dictionary[key]
```

مثلاً:

```
del student["age"]
```

یعنی:

آیتم دارای Key به نام ageرا حذف کن

۱۵- تفاوت delو ()pop

هر دو می‌توانند یک Key مشخص را حذف کنند

با :()pop

```
value = student.pop("age")
```

می‌توانیم مقدار حذف‌شده را دریافت کنیم

با :del

```
del student["age"]
```

مقدار حذف‌شده را دریافت نمی‌کنیم

۱۶- delبرای حذف کلDictionary

یکی از ویژگی‌های مهم delاین است که می‌توانیم خود Dictionary را کاملاً حذف کنیم

مثلاً:

```
car = {
   "brand": "Ford",
   "model": "Mustang",
   "year": 1964
}
del car
```

بعد از این دستور، متغیر Car دیگر وجود ندارد.

۱۷- تفاوت بسیار مهم delو ()clear

فرض کنید:

```
student = {
   "name": "Ali",
   "age": 22
}
```

اگر بنویسیم:

```
student.clear()
```

Dictionaryهنوز وجود دارد، ولی خالی است:

{}

اما اگر بنویسیم:

```
del student
```

خود متغیر studentحذف می‌شود

۱۸-روش چهارم()clear :

متد ()clear تمام آیتم‌های Dictionary را حذف می‌کن.

مثلاً:

```
car = {
   "brand": "Ford",
   "model": "Mustang",
   "year": 1964
}
car.clear()
print(car)
```

خروجی:

```
{}
```

۱۹- آیا clear()خود Dictionary را حذف می‌کند؟

خیر بعد از car.clear() متغیر carهمچنان وجود دارد فقط محتویات آن حذف شده‌اند.

```
print(car)
```

خروجی:

```
{}
```

بنابراین:

```
clear()
```

۲۰- مثال مقایسه‌ای بسیار مهم

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz"
}
```

حذف age

```
student.pop("age")
```

نتیجه:

```
{
   "name": "Ali",
   "city": "Shiraz"
}
```

حذف city

```
del student["city"]
```

نتیجه:

```
{
   "name": "Ali"
}
```

خالی کردنDictionary

```
student.clear()
```

نتیجه:

```
{}
```

حذف کاملDictionary

```
del student
```

۲۱- جدول کامل چهار روش

روش	چه چیزی حذف می‌شود؟	Dictionaryباقی می‌ماند؟	مقدار حذف‌شده برمی‌گردد؟
pop("key")	Keyمشخص	بله	بله
popitem()	آخرین آیتم	بله	بله
del d["key"]	Keyمشخص	بله	خیر
clear()	همه آیتم‌ها	بله	خیر
del d	کلDictionary	خیر	خیر

۲۲- مثال: حذف اطلاعات دانشجو

فرض کنید:

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz",
   "major": "Computer Engineering"
}
```

دانشجو شهر خود را تغییر داده و می‌خواهیم اطلاعات قبلی را حذف کنیم:

```
student.pop("city")
```

نتیجه:

```
{
   'name': 'Ali',
   'age': 22,
   'major': 'Computer Engineering'
}
```

۲۳-حذف با شرط if

می‌توانیم قبل از حذف بررسی کنیم که Key وجود دارد یا خیر

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz"
}
if "city" in student:
   student.pop("city")
```

```
print(student)
```

چرا این کار خوب است؟ چون اگر cityوجود نداشته باشد، برنامه با KeyErrorمتوقف نمی‌شود

۲۴-ترکیب inو del

همین کار را با delهم می‌توانیم انجام دهیم:

```
if "city" in student:
    del student["city"]
```

این الگو در برنامه‌های واقعی بسیار کاربردی است

۲۵- مثال کاربردی باInput

فرض کنید کاربر می‌خواهد یک Key را حذف کند:

```
student = {
    "name": "Ali",
    "age": 22,
    "city": "Shiraz",
    "major": "Computer Engineering"
}
key = input("Enter key to remove: ")
if key in student:
    student.pop(key)
    print("Item removed.")
else:
    print("Key not found.")
print(student)
```

مثلاً اگر کاربر وارد کند:

```
city
```

خروجی:

Item removed

۲۶- پروژه کوچک: مدیریت اطلاعات دانشجو

در این پروژه دانشجو باید بتواند یک اطلاعات را حذف کند.

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz",
   "major": "Computer Engineering"
}
print("Student Information:")
print(student)
key = input("Which information do you want to remove? ")
if key in student:
   removed = student.pop(key)
   print("Removed value:", removed)
else:
   print("This key does not exist.")
print("Updated Student:")
print(student)
```

۲۷- مثال فروشگاه

فرض کنید:

```
product = {
   "name": "Laptop",
   "brand": "Dell",
```

```
    "price": 1200,
    "stock": 10,
    "color": "Black"
}
```

اگر محصول دیگر رنگ نداشته باشد:

```
product.pop("color")
```

اگر بخواهیم آخرین اطلاعات اضافه‌شده را حذف کنیم:

```
product.popitem()
```

اگر بخواهیم کل اطلاعات محصول را خالی کنیم:

```
product.clear()
```

و اگر دیگر اصلاً به متغیر محصول نیاز نداشته باشیم:

```
del product
```

۲۸-یک اشتباه رایج

دانشجو ممکن است بنویسد:

```
student.pop()
```

اما این دستور برای حذف یک Key مشخص کافی نیست باید Key را مشخص کنیم:

```
student.pop("age")
```

در مقابل:

```
student.popitem()
```

نیازی به Key ندارد

۲۹- نکته مهم درباره ()popitem

در نسخه‌های جدیدPython ، Dictionaryترتیب ورود آیتم‌ها را حفظ می‌کند و ()popitemآخرین آیتم واردشده را حذف می‌کند

مثلاً:

```
data = {}
data["a"] = 10
data["b"] = 20
data["c"] = 30
```

آخرین آیتم:

c → 30

بنابراین:

```
data.popitem()
```

باعث حذف c → 30 می‌شود

۳۰- مثال با چند بار popitem()

```
data = {
   "a": 10,
   "b": 20,
   "c": 30
}
print(data.popitem())
print(data.popitem())
print(data)
```

خروجی:

```
('c', 30)
('b', 20)
{'a': 10}
```

بنابراین popitem()می‌تواند برای پردازش آیتم‌ها از انتهای Dictionary نیز استفاده شود

۳۱- یک مثال جالب: حذف تا خالی شدن Dictionary

```
data = {
   "a": 10,
   "b": 20,
   "c": 30
}
while data:
   item = data.popitem()
   print(item)
print(data)
```

خروجی:

```
('c', 30)
('b', 20)
('a', 10)
{}
```

۳۲-مقایسه نهایی با یک سناریو

فرض کنید:

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz",
   "gpa": 3.5
}
```

می‌خواهیم city را حذف کنیم:

```
student.pop("city")
```

می‌خواهیم آخرین آیتم را حذف کنیم:

```
student.popitem()
```

می‌خواهیم ageرا حذف کنیم:

```
del student["age"]
```

می‌خواهیم تمام اطلاعات را حذف کنیم ولی Dictionary باقی بماند:

```
student.clear()
```

می‌خواهیم خود Dictionary را حذف کنیم:

```
del student
```

۳۶-تمرین‌های کلاسی

تمرین ۱

Dictionaryزیر را ایجاد کنید:

```
car = {
   "brand": "BMW",
   "model": "X5",
   "year": 2025,
   "color": "Black"
}
```

با استفاده از ()popمقدار colorرا حذف کنید

تمرین ۲

Dictionaryزیر را ایجاد کنید:

```
student = {
   "name": "Ali",
   "age": 22,
   "city": "Shiraz",
```

```
    "major": "Computer Engineering"
}
```

با استفاده از del، مقدار ageرا حذف کنید

تمرین ۳

Dictionaryزیر را ایجاد کنید:

```
product = {
    "name": "Laptop",
    "brand": "Dell",
    "price": 1200,
    "stock": 10
}
```

با استفاده از popitem()آخرین آیتم را حذف کنید

تمرین ۴

Dictionaryزیر را ایجاد کنید:

```
student = {
    "name": "Ali",
    "age": 22,
    "city": "Shiraz"
}
```

با استفاده از clear()تمام آیتم‌ها را حذف کنید و سپس Dictionary را چاپ کنید

خروجی مورد انتظار:

```
{}
```

تمرین ۵

برنامه‌ای بنویسید که:

۱. یک Dictionary شامل اطلاعات دانشجو ایجاد کند

۲. Keyموردنظر را از کاربر دریافت کند

۳. بررسی کند Key وجود دارد یا خیر

۴. اگر وجود داشت آن را با ()popحذف کند

۵. اگر وجود نداشت، پیام مناسب نمایش دهد

تمرین ۶

برنامه‌ای برای مدیریت محصول بنویسید:

```
product = {
   "name": "Laptop",
   "brand": "Lenovo",
   "price": 1500,
   "stock": 20,
   "color": "Gray"
}
```

از کاربر بپرسید Which item do you want to remove? اگر Key وجود داشت آن را حذف کنید و Dictionary جدید را نمایش دهید

تمرین ۷

برنامه‌ای بنویسید که Dictionary زیر را:

```
data = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz",
```

```
    "country": "Turkey"
}
```

به کاربر نمایش دهد و از او بپرسد:

```
1. Remove an item
2. Remove last item
3. Clear dictionary
4. Exit
```

Loop Through a Dictionary

۱- چرا باید روی Dictionary Loop بزنیم؟

فرض کنید Dictionary زیر را داریم:

```
student = {
    "name": "Ali",
    "age": 23,
    "city": "Shiraz",
    "major": "Computer Engineering"
}
```

اگر بخواهیم فقط یک مقدار را ببینیم، می‌توانیم بنویسیم:

```
print(student["name"])
```

اما اگر بخواهیم **تمام اطلاعات** را نمایش دهیم، نوشتن:

```
print(student["name"])
print(student["age"])
print(student["city"])
print(student["major"])
```

روش مناسبی نیست

اینجاست که forوارد می‌شود:

```
for x in student:
  print(x)
```

۲- Loop ساده روی Dictionary

ساده‌ترین شکل:

```
for x in student:
  print(x)
```

خروجی:

```
name
age
city
major
```

نکته بسیار مهم

وقتی مستقیماً روی Dictionary حلقه می‌زنیم:

```
for x in student:
```

متغیر xبه صورت پیش‌فرض **Key**ها را دریافت می‌کند

یعنی:

```
x = "name"
x = "age"
x = "city"
x = "major"
```

۳- نمایش Key ها

مثال:

```
car = {
```

```
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
for key in car:
    print(key)
```

خروجی:

```
brand
model
year
```

در اینجا:

```
for key in car:
```

یعنی:

تک‌تکKey های Dictionary را در اختیار من قرار بده

۴- نمایشValue ها

اگر بخواهیم به جایKey ، Valueها را چاپ کنیم:

```
student = {
    "name": "Ali",
    "age": 23,
    "city": "Shiraz"
}
for key in student:
    print(student[key])
```

خروجی:

```
Ali
```

```
23
Shiraz
```

اینجا باید به این نکته دقت کنیم:

```
student[key]
```

یعنی Valueمربوط به Key فعلی را بده

۶- استفاده از values()

روش دیگری برای دریافتValue ها استفاده از values() است

مثلاً:

```
student = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
for value in student.values():
   print(value)
```

خروجی:

```
Ali
23
Shiraz
```

۷- تفاوت دو روش نمایشValue

روش اول

```
for key in student:
   print(student[key])
```

روش دوم

```
for value in student.values():
    print(value)
```

هر دو صحیح هستند اما وقتی فقطValue ها را می‌خواهیم، روش زیر خواناتر است:

```
for value in student.values():
```

۱۰- استفاده از ()keys

اگر هدف ما دریافتKey ها باشد، می‌توانیم از ()keys استفاده کنیم

```
student = {
    "name": "Ali",
    "age": 23,
    "city": "Shiraz"
}
for key in student.keys():
    print(key)
```

خروجی:

```
name
age
city
```

۱۰- مقایسه for dictو ()keys

این دو کد:

```
for key in student:
    print(key)
```

و:

```
for key in student.keys():
    print(key)
```

هر دوKey ها را پیمایش می‌کنند

بنابراین در حالت معمول:

```
for key in student:
```

کوتاه‌تر است

اما student.keys() به صورت واضح نشان می‌دهد که هدف ماKey ها هستند

۱۲- استفاده از items()

اگر بخواهیم **هم Key و هم Value** را همزمان داشته باشیم، از items() استفاده می‌کنیم

مثلاً:

```
student = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
for key, value in student.items():
   print(key, value)
```

خروجی:

```
name Ali
age 23
city Shiraz
```

14.چرا دو متغیر داریم؟

در:

```
for key, value in student.items():
```

دو متغیر داریم:

key → Key

value → Value

مثلاً در دور اول:

key = "name"

value = "Ali"

در دور دوم:

key = "age"

value = 23

و در دور سوم:

key = "city"

value = "Shiraz"

۱۵-نمایش زیباتر اطلاعات

می‌توانیم از f-string استفاده کنیم:

```
student = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
for key, value in student.items():
   print(f"{key}: {value}")
```

خروجی:

```
name: Ali
age: 23
city: Shiraz
```

۱۷- استفاده از if داخل Loop

فرض کنید:

```
student = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz",
   "gpa": 3.5
}
```

می‌خواهیم فقط اطلاعاتی را نمایش دهیم که مقدار آنها عددی است برای مثال ساده‌تر، می‌توانیم فقط gpa را پیدا کنیم:

```
for key, value in student.items():
   if key == "gpa":
      print(value)
```

خروجی:

```
3.5
```

۱۰- محاسبه مجموع Value ها

اگر Value ها عدد باشند، می‌توانیم روی آنها Loop بزنیم

```
scores = {
   "math": 18,
   "python": 19,
   "database": 16
}
total = 0
for score in scores.values():
   total += score
```

```
print(total)
```

خروجی:

```
53
```

۲۳- استفاده از for برای تغییر Dictionary

مثلاً:

```
prices = {
   "laptop": 1000,
   "phone": 500,
   "tablet": 300
}
```

می‌توانیم قیمت‌ها را افزایش دهیم:

```
for product in prices:
   prices[product] = prices[product] * 1.1
print(prices)
```

در این مثال:

```
prices[product]
```

Value مربوط به Key فعلی را تغییر می‌دهد

۲۴- یک نکته مهم درباره تغییر Dictionary هنگام Loop

برای دانشجویان مبتدی بهتر است این نکته را به صورت ساده بیان کنید:

هنگام Loop زدن، تغییر **Value‌ها** معمولاً مشکلی ایجاد نمی‌کند:

```
for key in prices:
   prices[key] = prices[key] * 1.1
```

اما اضافه یا حذف کردن Keyها در همان Loop می‌تواند باعث خطا شود

مثلاً:

```
for key in student:
    del student[key]
```

می‌تواند باعث RuntimeError: dictionary changed size during iteration شود پس بهتر است فعلاً هنگام Loop روی Dictionary ، ساختار Key ها را تغییر ندهیم.

۲۵- مثال: فروشگاه

فرض کنید:

```
products = {
    "Laptop": 1200,
    "Phone": 800,
    "Tablet": 500
}
```

می‌خواهیم قیمت‌ها را نمایش دهیم:

```
for product, price in products.items():
    print(f"{product}: ${price}")
```

خروجی:

```
Laptop: $1200
Phone: $800
Tablet: $500
```

۲۶-پیدا کردن محصولات گران

```
for product, price in products.items():
    if price > 700:
        print(product)
```

خروجی:

```
Laptop
Phone
```

یا:

```
for product, price in products.items():
    if price > 700:
        print(f"{product}: ${price}")
```

خروجی:

```
Laptop: $1200
Phone: $800
```

۲۸- مثال جامع

```
student = {
    "name": "Ali",
    "age": 23,
    "major": "Computer Engineering",
    "math": 18,
    "python": 19,
    "database": 16
}
print("Student Information:")
for key, value in student.items():
    print(f"{key}: {value}")
```

۳۰-پروژه کوچکStudent Management :

یک Dictionary برای دانشجو ایجاد کنید:

```
student = {
```

```
    "name": "Ali",
    "age": 23,
    "major": "Computer Engineering",
    "gpa": 3.5
}
```

سپس:

```
for key, value in student.items():
    print(f"{key}: {value}")
```

بعد بررسی کنید:

```
if student["gpa"] >= 3:
    print("Good Student")
else:
    print("Needs Improvement")
```

تمرین ۱

Dictionaryزیر را ایجاد کنید:

```
student = {
    "name": "Ali",
    "age": 23,
    "city": "Shiraz"
}
```

با استفاده از forتمامKey ها را چاپ کنید

تمرین ۲

Dictionaryتمرین قبل را استفاده کنید و تمامValue ها را با ()valuesچاپ کنید

تمرین ۳

با استفاده از items()خروجی زیر را ایجاد کنید:

name: Ali

age: 23

city: Shiraz

تمرین ۴

Dictionaryزیر را در نظر بگیرید:

```
scores = {
   "math": 18,
   "python": 19,
   "database": 16,
   "english": 17
}
```

با استفاده از values()مجموع نمرات را محاسبه کنید

تمرین ۵

میانگین نمرات Dictionary زیر را محاسبه کنید:

```
scores = {
   "math": 18,
   "python": 19,
   "database": 16,
   "english": 17
}
```

تمرین ۶

با استفاده از items()بالاترین نمره را پیدا کنید و نام درس را نیز نمایش دهید

خروجی مورد انتظار:

Best subject: python

Score: 19

تمرین ۷

Dictionaryزیر را در نظر بگیرید:

```
products = {
   "Laptop": 1200,
   "Phone": 800,
   "Tablet": 500,
   "Mouse": 30
}
```

با استفاده از forو items()فقط محصولاتی را چاپ کنید که قیمت آنها بیشتر از 700است

تمرین ۸

یک Dictionary برای یک دانشجو ایجاد کنید:

```
student = {
   "name": "Ali",
   "math": 18,
   "python": 19,
   "database": 16,
   "english": 17
}
```

برنامه‌ای بنویسید که:

۱. نام دانشجو را نمایش دهد

۲. تمام نمرات را نمایش دهد

۳. مجموع نمرات را محاسبه کند

۴. میانگین را محاسبه کند

۵. بالاترین نمره را پیدا کند

۶. نام درسی که بالاترین نمره را دارد نمایش دهد

۷. اگر میانگین بیشتر یا مساوی 17بود، چاپ کند :

Excellent Student

در غیر این صورت:

Keep Practicing

Copy Dictionaries

مشکل اصلی چیست؟

فرض کنید Dictionary زیر را داریم:

```
student1 = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
```

ممکن است بخواهیم یک نسخه دیگر از آن بسازیم:

```
student2 = student1
```

در نگاه اول شاید فکر کنیم:

student1 → Dictionaryاصلی

student2 → Dictionaryکپی

اما این تصور اشتباه است

۴- مثال

```
student1 = {
   "name": "Ali",
```

```
    "age": 23
}
student2 = student1
student1["age"] = 30
print(student1)
print(student2)
```

خروجی:

```
{'name': 'Ali', 'age': 30}
{'name': 'Ali', 'age': 30}
```

۵- یک مثال دیگر

```
student1 = {
    "name": "Ali",
    "age": 23
}
student2 = student1
student2["name"] = "Reza"
print(student1)
print(student2)
```

خروجی:

```
{'name': 'Reza', 'age': 23}
{'name': 'Reza', 'age': 23}
```

یعنی تغییر از طریق student2نیز روی student1تأثیر گذاشت

۶- نکته مهم = :در اینجا Copy نیست

این خط: student2 = student1

به معنای از Dictionary اول یک نسخه جدید بساز نیست

بلکه به صورت ساده می‌توان گفت:

نام student2را نیز به همان Dictionary متصل کن

۷- روش صحیح اول()copy :

برای ساختن یک کپی مستقل می‌توانیم از ()copyاستفاده کنیم

مثلاً:

```
student1 = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
student2 = student1.copy()
print(student2)
```

خروجی:

```
{'name': 'Ali', 'age': 23, 'city': 'Shiraz'}
```

۸-آیا این بار تغییرات منتقل می‌شوند؟

خیر

```
student1 = {
   "name": "Ali",
   "age": 23
}
student2 = student1.copy()
student1["age"] = 30
print(student1)
print(student2)
```

خروجی:

```
{'name': 'Ali', 'age': 30}
{'name': 'Ali', 'age': 23}
```

۱۱-تغییر Dictionary کپی

```
student1 = {
   "name": "Ali",
   "age": 23
}
student2 = student1.copy()
student2["city"] = "Shiraz"
print(student1)
print(student2)
```

خروجی:

```
{'name': 'Ali', 'age': 23}
{'name': 'Ali', 'age': 23, 'city': 'Shiraz'}
```

بنابراین تغییرات مستقل هستند

۱۲- روش دوم()dict :

روش دیگری برای ساخت کپی Dictionary استفاده از تابع ()dict است

مثلاً:

```
student1 = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
```

```
student2 = dict(student1)
print(student2)
```

خروجی:

```
{'name': 'Ali', 'age': 23, 'city': 'Shiraz'}
```

۱۳- آیا dict()نیز کپی ایجاد می‌کند؟

بله، در این کاربرد یک Dictionary جدید ایجاد می‌کند

مثلاً:

```
student1 = {
   "name": "Ali",
   "age": 23
}
student2 = dict(student1)
student1["age"] = 30
print(student1)
print(student2)
```

خروجی:

```
{'name': 'Ali', 'age': 30}
{'name': 'Ali', 'age': 23}
```

۱۶-بررسی با is

Pythonعملگر is را نیز دارد که می‌توان از آن برای بررسی اینکه دو متغیر به همان شیء اشاره می‌کنند یا نه استفاده کرد

مثلاً:

```
student1 = {
   "name": "Ali"
}
```

```
student2 = student1
print(student1 is student2)
```

خروجی: True ، چون هر دو به همان Dictionary اشاره می‌کنند

۱۷-استفاده از ()copy

```
student1 = {
   "name": "Ali"
}
student2 = student1.copy()
print(student1 is student2)
```

خروجی False ، چون دو Dictionary مستقل داریم

۱۸- == و isرا اشتباه نگیرید

فرض کنید:

```
a = {
   "name": "Ali"
}
b = a.copy()
```

اگر بنویسیم:

```
print(a == b)
```

خروجی:

```
True
```

چون محتوای آنها برابر است اما print(a is b)

خروجی:

False

چون خود شیء یکسان نیست

بنابراین:

== → آیا محتوا برابر است؟

is → آیا همان Object است؟

۱۹- مثال :

```
a = {
   "name": "Ali",
   "age": 23
}
b = a.copy()
print(a == b)
print(a is b)
```

خروجی:

```
True
False
```

این مثال تفاوت برابری محتوا و یکسان بودن Object را به خوبی نشان می‌دهد

۲۰- یک نکته مهم‌تر Dictionary :تو در تو

تا اینجا مثال‌ها ساده بودند:

```
student = {
   "name": "Ali",
   "age": 23
}
```

اما ممکن است Dictionary شامل List یا Dictionary دیگری باشد

مثلاً:

```
student1 = {
   "name": "Ali",
   "skills": ["Python", "Django"]
}
```

اگر بنویسیم:

```
student2 = student1.copy()
```

یک Shallow Copyایجاد می‌شود

()copyبرای ساختارهای تو در تو، کپی عمیق همه‌ی داده‌های داخلی نیست

مثلاً:

```
student1 = {
   "name": "Ali",
   "skills": ["Python", "Django"]
}
student2 = student1.copy()
student1["skills"].append("SQL")
print(student1)
print(student2)
```

ممکن است هر دوDictionary ، SQLرا در skillsببینند این موضوع مربوط به **Shallow Copy**است و در مباحث پیشرفته‌تر می‌توان آن را با ()deepcopyبررسی کرد

۲۲-مثال: نسخه اولیه و نسخه جدید

فرض کنید یک محصول داریم:

```
product = {
   "name": "Laptop",
   "price": 1200,
   "stock": 10
```

```
}
```

می‌خواهیم قبل از اعمال تغییرات، یک نسخه از اطلاعات آن نگه داریم:

```
old_product = product.copy()
product["price"] = 1100
print("Old:", old_product)
print("New:", product)
```

خروجی:

```
Old: {'name': 'Laptop', 'price': 1200, 'stock': 10}
New: {'name': 'Laptop', 'price': 1100, 'stock': 10}
```

این مفهوم در برنامه‌های واقعی بسیار کاربردی است

۲۳- مثال کاربردی: ایجاد نسخه دانشجو

```
student = {
   "name": "Ali",
   "age": 23,
   "major": "Computer Engineering"
}
student_backup = student.copy()
student["age"] = 24
print("Original:")
print(student)
print("Backup:")
print(student_backup)
```

خروجی:

```
Original:
{'name': 'Ali', 'age': 24, 'major': 'Computer Engineering'}
```

Backup:

{'name': 'Ali', 'age': 23, 'major': 'Computer Engineering'}

۲۴- پروژه

Student Information Backup

برنامه‌ای بنویسید که:

۱. اطلاعات یک دانشجو را در Dictionary ذخیره کند

۲. از Dictionary یک Copy ایجاد کند

۳. اطلاعات اصلی را تغییر دهد

۴. هر دو Dictionary را نمایش دهد

۵. نشان دهد که اطلاعات Copy تغییر نکرده است

نمونه:

```
student = {
   "name": "Ali",
   "age": 23,
   "major": "Computer Engineering"
}
backup = student.copy()
student["age"] = 24
student["major"] = "Data Science"
print("Original:")
print(student)
print("Backup:")
print(backup)
```

تمرین ۱

Dictionaryزیر را ایجاد کنید:

```
car = {
  "brand": "BMW",
  "model": "X5",
  "year": 2025
}
```

با استفاده از copy()یک نسخه از آن بسازید

تمرین ۲

Dictionaryزیر را ایجاد کنید:

```
student1 = {
  "name": "Ali",
  "age": 23
}
```

یک Dictionary به نام student2با استفاده از copy()ایجاد کنید

سپس:

```
student1["age"] = 30
```

را اجرا کنید . بررسی کنید آیا student2تغییر کرده است یا خیر

تمرین ۳

همین تمرین را با dict()انجام دهید

تمرین ۴

کد زیر را اجرا کنید:

```
a = {
  "name": "Ali"
```

```
}
b = a
a["name"] = "Reza"
print(a)
print(b)
```

توضیح دهید چرا مقدار bنیز تغییر کرده است

تمرین ۵

کد زیر را اجرا کنید:

```
a = {
   "name": "Ali"
}
b = a.copy()
a["name"] = "Reza"
print(a)
print(b)
```

توضیح دهید چرا این بار bتغییر نکرده است

تمرین ۶

خروجی کد زیر را قبل از اجرای آن پیش‌بینی کنید:

```
a = {
   "name": "Ali",
   "age": 23
}
b = a.copy()
print(a == b)
print(a is b)
```

سپس برنامه را اجرا کنید و پاسخ خود را بررسی کنید

تمرین ۷

برنامه‌ای برای مدیریت محصول بنویسید:

```
product = {
   "name": "Laptop",
   "price": 1200,
   "stock": 10
}
```

یک Backup از Dictionary ایجاد کنید

سپس قیمت محصول را به 1000تغییر دهید

در نهایت هر دو Dictionary را نمایش دهید:

```
Original:
...
Backup:
...
```

Nested Dictionaries

Nested Dictionaryچیست؟

کلمه‌ی **Nested**یعنی تودرتو بنابراین:

Nested Dictionaryیعنی یک Dictionary که داخل خودش یک یا چند Dictionary دیگر دارد

یک Dictionary معمولی:

```
student = {
   "name": "Ali",
   "age": 23
}
```

اما Nested Dictionary می‌تواند به این شکل باشد:

```
students = {
   "student1": {
      "name": "Ali",
      "age": 23
   },
   "student2": {
      "name": "Reza",
      "age": 25
   }
}
```

۳- چرا از Nested Dictionary استفاده می‌کنیم؟

فرض کنید می‌خواهیم اطلاعات سه دانشجو را ذخیره کنیم اگر ازDictionary های جدا استفاده کنیم:

```
student1 = {
   "name": "Ali",
   "age": 23
}
student2 = {
   "name": "Reza",
   "age": 25
}
student3 = {
   "name": "Sara",
   "age": 22
}
```

حالا سه Dictionary مستقل داریم اما می‌توانیم همه را داخل یک Dictionary قرار دهیم:

```
students = {
   "student1": student1,
   "student2": student2,
   "student3": student3
}
```

این ساختار مدیریت اطلاعات را راحت‌تر می‌کند

۴- ساختNested Dictionary

می‌توانیم مستقیماًDictionary های داخلی را داخل Dictionary اصلی بنویسیم:

```
myfamily = {
   "child1": {
      "name": "Emil",
      "year": 2004
   },
   "child2": {
      "name": "Tobias",
      "year": 2007
   },
   "child3": {
      "name": "Linus",
      "year": 2011
   }
}
```

۵- Dictionary بیرونی و داخلی

برای درک بهتر، دو مفهوم داریم:

Outer Dictionary

Dictionaryاصلی:

myfamily

Inner Dictionary

Dictionaryهایی که داخل آن قرار دارند:

myfamily["child1"]

myfamily["child2"]

myfamily["child3"]

مثلاً:

```
print(myfamily["child1"])
```

خروجی:

```
{'name': 'Emil', 'year': 2004}
```

۶- دسترسی به Dictionary داخلی

اگر بخواهیم اطلاعات child2را بگیریم:

```
print(myfamily["child2"])
```

خروجی:

```
{'name': 'Tobias', 'year': 2007}
```

اما اگر فقط نام او را بخواهیم:

```
print(myfamily["child2"]["name"])
```

خروجی:

```
Tobias
```

۷- چرا دو [ ]داریم؟

myfamily["child2"]["name"]

از چپ به راست خوانده می‌شود

ابتدا:

```
myfamily["child2"]
```

Dictionaryمربوط به child2را پیدا می‌کند:

```
{
  "name": "Tobias",
  "year": 2007
}
```

سپس:

```
["name"]
```

از داخل آن Dictionary مقدار nameرا پیدا می‌کند

پس:

```
myfamily["child2"]["name"]
```

۸- دسترسی به سن

```
print(myfamily["child3"]["year"])
```

خروجی:

```
2011
```

۹-مثال دانشجویان

```
students = {
  "student1": {
    "name": "Ali",
    "age": 23,
    "major": "Computer Engineering"
  },
```

```
  "student2": {
    "name": "Reza",
    "age": 25,
    "major": "Data Science"
  },
  "student3": {
    "name": "Sara",
    "age": 22,
    "major": "Software Engineering"
  }
}
```

۱۰- دسترسی به نام دانشجو

نام دانشجوی اول:

```
print(students["student1"]["name"])
```

خروجی:

```
Ali
```

نام دانشجوی دوم:

```
print(students["student2"]["name"])
```

خروجی:

```
Reza
```

نام دانشجوی سوم:

```
print(students["student3"]["name"])
```

خروجی:

```
Sara
```

۱۱- دسترسی به رشته تحصیلی

```
print(students["student1"]["major"])
```

خروجی:

```
Computer Engineering
```

۱۲-تغییر اطلاعات داخلNested Dictionary

Nested Dictionaryنیز قابل تغییر است مثلاً سن Ali را تغییر دهیم:

```
students["student1"]["age"] = 24
```

حالا:

```
print(students["student1"])
```

خروجی:

```
{
   'name': 'Ali',
   'age': 24,
   'major': 'Computer Engineering'
}
```

تغییر رشته تحصیلی

```
students["student2"]["major"] = "Artificial Intelligence"
```

۱۴- اضافه کردن اطلاعات جدید

فرض کنید می‌خواهیم GPA دانشجوی اول را اضافه کنیم:

```
students["student1"]["gpa"] = 3.5
```

حالا:

```
print(students["student1"])
```

خروجی:

```
{
   'name': 'Ali',
   'age': 24,
   'major': 'Computer Engineering',
   'gpa': 3.5
}
```

۱۵- حذف اطلاعات داخلی

مثلاً می‌خواهیم ageرا حذف کنیم:

```
students["student1"].pop("age")
```

یا:

```
del students["student1"]["age"]
```

۱۶- اضافه کردن یک Dictionary جدید

می‌توانیم یک دانشجوی جدید اضافه کنیم:

```
students["student4"] = {
   "name": "Mina",
   "age": 24,
   "major": "Computer Science"
}
```

حالا چهار دانشجو داریم

۱۷-ایجاد Nested Dictionary باDictionary های جدا

ابتدا:

```
student1 = {
   "name": "Ali",
   "age": 23
}
```

```
student2 = {
   "name": "Reza",
   "age": 25
}
student3 = {
   "name": "Sara",
   "age": 22
}
```

سپس:

```
students = {
   "student1": student1,
   "student2": student2,
   "student3": student3
}
```

حالا print(students) اطلاعات هر سه دانشجو را در خود دارد

۱۸-تفاوت دو روش ایجادNested Dictionary

روش اول

همه‌چیز را مستقیم می‌نویسیم:

```
students = {
   "student1": {
      "name": "Ali",
      "age": 23
   },
   "student2": {
```

```
    "name": "Reza",
    "age": 25
  }
}
```

روش دوم

ابتداDictionary های داخلی را ایجاد می‌کنیم:

```
student1 = {
  "name": "Ali",
  "age": 23
}
student2 = {
  "name": "Reza",
  "age": 25
}
students = {
  "student1": student1,
  "student2": student2
}
```

هر دو روش صحیح هستند روش اول برای داده‌های کوچک ساده‌تر است روش دوم زمانی مفید است که بخواهیم Dictionaryهای داخلی را جداگانه مدیریت کنیم.

۱۹- Loop روی Nested Dictionary

فرض کنید:

```
students = {
  "student1": {
    "name": "Ali",
```

```
    "age": 23
  },
  "student2": {
    "name": "Reza",
    "age": 25
  }
}
```

اگر بنویسیم:

```
for key, value in students.items():
  print(key)
```

خروجی:

```
student1
student2
```

۲۰- دریافت Dictionary داخلی

اگر بنویسیم:

```
for key, value in students.items():
  print(value)
```

خروجی:

```
{'name': 'Ali', 'age': 23}
{'name': 'Reza', 'age': 25}
```

یعنی valueدر اینجا خودش یک Dictionary است

۲۱- دو حلقه for

برای دسترسی به اطلاعات داخلی می‌توانیم از دو Loop استفاده کنیم:

```
for student_id, student in students.items():
  print(student_id)
```

```
    for key, value in student.items():
        print(key, value)
```

خروجی:

```
student1
name Ali
age 23
student2
name Reza
age 25
```

۲۵- استفاده از if در Nested Dictionary

مثلاً می‌خواهیم دانشجویان بالای ۲۳ سال را پیدا کنیم:

```
for student_id, student in students.items():
    if student["age"] > 23:
        print(student["name"])
```

خروجی:

```
Reza
```

۲۶- مثال مهم: پیدا کردن دانشجوی رشته خاص

```
for student_id, student in students.items():
    if student["major"] == "Data Science":
        print(student["name"])
```

خروجی:

```
Reza
```

۲۷- مثال: نمایش فقط نام دانشجویان

لازم نیست همیشه از دو Loop استفاده کنیم اگر فقط نام دانشجویان را می‌خواهیم:

```
for student in students.values():
   print(student["name"])
```

خروجی:

```
Ali
Reza
Sara
```

۲۸- مثال: محاسبه میانگینGPA

فرض کنید:

```
students = {
   "student1": {
      "name": "Ali",
      "gpa": 3.5
   },
   "student2": {
      "name": "Reza",
      "gpa": 3.8
   },
   "student3": {
      "name": "Sara",
      "gpa": 3.2
   }
}
```

میانگین:

```
total = 0
for student in students.values():
    total += student["gpa"]
average = total / len(students)
print(f"Average GPA: {average:.2f}")
```

خروجی:

```
Average GPA: 3.50
```

۲۹- مثال: سیستم اطلاعات محصولات

```
products = {
    "product1": {
        "name": "Laptop",
        "price": 1200,
        "stock": 10
    },
    "product2": {
        "name": "Phone",
        "price": 800,
        "stock": 20
    },
    "product3": {
        "name": "Tablet",
        "price": 500,
        "stock": 15
    }
}
```

نمایش محصولات:

```
for product_id, product in products.items():
    print(
        f"{product_id}: "
        f"{product['name']} - "
        f"${product['price']}"
    )
```

خروجی:

```
product1: Laptop - $1200
product2: Phone - $800
product3: Tablet - $500
```

۳۰- پیدا کردن محصولات گران

```
for product in products.values():
    if product["price"] > 700:
        print(product["name"])
```

خروجی:

```
Laptop
Phone
```

۳۱- تغییر قیمت محصولات

فرض کنید ۱۰ درصد تخفیف داریم:

```
for product in products.values():
    product["price"] *= 0.9
```

حالا قیمت‌ها ۱۰ درصد کاهش پیدا کرده‌اند

۳۲- Nested Dictionary چندلایه

Dictionaryحتی می‌تواند بیشتر از دو سطح داشته باشد

مثلاً:

```
company = {
   "employee1": {
      "name": "Ali",
      "contact": {
         "email": "ali@example.com",
         "phone": "123456"
      }
   }
}
```

۳۳- یک نکته بسیار مهم

درNested Dictionary ، تعداد []ها نشان‌دهنده تعداد سطح دسترسی است

مثلاً:

```
data["student"]
```

یک سطح

```
data["student"]["name"]
```

دو سطح

```
data["student"]["contact"]["email"]
```

سه سطح

۳۵-پروژه کلاسیStudent Management :

یک سیستم ساده دانشجویان:

```
students = {
```

```
    "student1": {
        "name": "Ali",
        "age": 23,
        "major": "Computer Engineering",
        "gpa": 3.5
    },
    "student2": {
        "name": "Reza",
        "age": 25,
        "major": "Data Science",
        "gpa": 3.8
    },
    "student3": {
        "name": "Sara",
        "age": 22,
        "major": "Software Engineering",
        "gpa": 3.2
    }
}
```

نمایش:

```
for student_id, student in students.items():
    print(f"ID: {student_id}")
    print(f"Name: {student['name']}")
    print(f"Age: {student['age']}")
    print(f"Major: {student['major']}")
    print(f"GPA: {student['gpa']}")
```

```
print("----------------------")
```

تمرین ۱

Nested Dictionaryزیر را ایجاد کنید:

```
family = {
   "child1": {
      "name": "Ali",
      "age": 15
   },
   "child2": {
      "name": "Sara",
      "age": 12
   }
}
```

نام child1را چاپ کنید

تمرین ۲

سن child2را چاپ کنید

تمرین ۳

سن child1را به 16تغییر دهید

تمرین ۴

یک Child جدید به نام child3اضافه کنید

تمرین ۵

Dictionaryزیر را در نظر بگیرید:

```
students = {
   "student1": {
```

```
    "name": "Ali",
    "gpa": 3.5
  },
  "student2": {
    "name": "Reza",
    "gpa": 3.8
  },
  "student3": {
    "name": "Sara",
    "gpa": 3.2
  }
}
```

با استفاده از for نام تمام دانشجویان را چاپ کنید

تمرین ۶

با استفاده از for دانشجویانی را چاپ کنید که GPA آنها بیشتر از 3.5 است

خروجی:

Reza

تمرین ۷

میانگین GPA تمام دانشجویان را محاسبه کنید

تمرین ۸

با استفاده از ()items نام و GPA تمام دانشجویان را نمایش دهید:

Ali: 3.5

Reza: 3.8

Sara: 3.2

تمرین ۹

یک Nested Dictionary برای محصولات فروشگاه ایجاد کنید:

Laptop

Phone

Tablet

هر محصول باید حداقل شامل این اطلاعات باشد:

name

price

stock

سپس برنامه‌ای بنویسید که فقط محصولاتی را نمایش دهد که:

price > 700

۳۷- پروژه

Student Management System

برنامه‌ای طراحی کنید که اطلاعات حداقل ۵ دانشجو را در یک Nested Dictionary ذخیره کند

هر دانشجو باید داشته باشد:

name

age

major

math

python

database

مثلاً:

```
students = {
   "student1": {
      "name": "Ali",
```

```
    "age": 23,
    "major": "Computer Engineering",
    "math": 18,
    "python": 19,
    "database": 16
  },
  "student2": {
    "name": "Reza",
    "age": 24,
    "major": "Data Science",
    "math": 17,
    "python": 18,
    "database": 19
  }
}
```

برنامه باید بتواند:

۱. تمام دانشجویان را نمایش دهد
۲. نام یک دانشجوی مشخص را نمایش دهد
۳. معدل هر دانشجو را محاسبه کند
۴. بالاترین نمره Python را پیدا کند
۵. دانشجویانی که میانگین آنها بالاتر از 17است نمایش دهد
۶. اطلاعات یک دانشجو را تغییر دهد
۷. یک دانشجوی جدید اضافه کند

Python Dictionary Methods

Dictionaryیکی از مهم‌ترین ساختارهای داده در Python است و Python تعدادی **Method**آماده در اختیار ما قرار می‌دهد تا بتوانیم اطلاعات Dictionary را:

- مشاهده کنیم
- جست‌وجو کنیم
- اضافه کنیم
- تغییر دهیم
- حذف کنیم
- کپی کنیم
- مقداردهی اولیه کنیم

متدهای اصلی Dictionary عبارت‌اند از:

Method	کاربرد
clear()	حذف تمام عناصر
copy()	ایجاد یک کپی
fromkeys()	ساخت Dictionary باKey های مشخص
get()	دریافت مقدار یکKey
items()	دریافت Key وValue
keys()	دریافتKey ها
pop()	حذف یک عنصر بر اساسKey
popitem()	حذف آخرین جفتKey-Value
setdefault()	دریافت مقدار و در صورت نبودنKey ، ایجاد آن
update()	اضافه یا به‌روزرسانی عناصر
values()	دریافتValue ها

clear()-۱

متد clear()تمام عناصر Dictionary را حذف می‌کند

مثال

```
student = {
   "name": "Ali",
   "age": 23,
   "major": "Computer Engineering"
}
student.clear()
print(student)
```

خروجی:

```
{}
```

توجه کنید که خود Dictionary هنوز وجود دارد؛ فقط محتویات آن حذف شده‌اند.

پس student.clear() یعنی تمام عناصر را حذف کن اما del student یعنی خود متغیر studentرا حذف کن.

تفاوت

```
student.clear()
print(student)
```

خروجی:

```
{}
```

ولی:

```
del student
print(student)
```

خطا ایجاد می‌کند، چون دیگر studentوجود ندارد

۲-copy()

برای ایجاد یک کپی مستقل از Dictionary استفاده می‌شود

```
student = {
   "name": "Ali",
```

```
   "age": 23
}
student2 = student.copy()
print(student2)
```

خروجی:

```
{'name': 'Ali', 'age': 23}
```

حالا اگر Dictionary اصلی را تغییر دهیم:

```
student["age"] = 30
print(student)
print(student2)
```

خروجی:

```
{'name': 'Ali', 'age': 30}
{'name': 'Ali', 'age': 23}
```

بنابراین:

```
student2 = student.copy()
```

یک Dictionary جدید ایجاد می‌کند

نکته مهم : این student2 = student کپی مستقل نیست

ولی ()student2 = student.copy کپی ایجاد می‌کند

۳- fromkeys()

متد ()fromkeysبرای ساختن یک Dictionary با مجموعه‌ای ازKey ها استفاده می‌شود.

ساختار کلی:

```
dict.fromkeys(keys, value)
```

مثلاً:

```
keys = ("name", "age", "city")
```

```
student = dict.fromkeys(keys)
print(student)
```

خروجی:

```
{'name': None, 'age': None, 'city': None}
```

اگر مقدار اولیه مشخص کنیم:

```
keys = ("name", "age", "city")
student = dict.fromkeys(keys, "Unknown")
print(student)
```

خروجی:

```
{'name': 'Unknown', 'age': 'Unknown', 'city': 'Unknown'}
```

مثال :

فرض کنید می‌خواهیم اطلاعات ۴ درس را ایجاد کنیم، ولی هنوز نمره‌ای ثبت نشده است:

```
subjects = ["Python", "Database", "Math", "English"]
grades = dict.fromkeys(subjects, 0)
print(grades)
```

خروجی:

```
{'Python': 0, 'Database': 0, 'Math': 0, 'English': 0}
```

این کاربرد برای مقداردهی اولیه بسیار مناسب است

۴-get()

یکی از مهم‌ترین متدهای Dictionary است برای دریافت Value مربوط به یک Key استفاده می‌شود

```
student = {
   "name": "Ali",
   "age": 23
```

```
}
print(student.get("name"))
```

خروجی:

```
Ali
```

تفاوت get() و []

اگر بنویسیم print(student["name"]) مقدار را دریافت می‌کنیم اما اگر Key وجود نداشته باشد:

```
print(student["city"])
```

خطای KeyError ایجاد می‌شود.

ولی:

```
print(student.get("city"))
```

خروجی:

None است

مقدار پیش‌فرض با get()

یکی از ویژگی‌های بسیار مفید get()این است که می‌توانیم مقدار پیش‌فرض تعیین کنیم:

```
print(student.get("city", "Not Found"))
```

خروجی:

```
Not Found
```

مثلاً:

```
print(student.get("age", 0))
```

اگر ageوجود داشته باشد، مقدار آن را برمی‌گرداند

اگر وجود نداشته باشد:

0

را برمی‌گرداند

۵-items()

متد items()تمام جفت‌های Key + Value را در اختیار ما قرار می‌دهد.

مثلاً:

```
student = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
print(student.items())
```

خروجی شبیه زیر است :

```
dict_items([
   ('name', 'Ali'),
   ('age', 23),
   ('city', 'Shiraz')
])
```

استفاده از items()با for

```
for key, value in student.items():
   print(key, value)
```

خروجی:

```
name Ali
age 23
city Shiraz
```

اگر بخواهیم تمام اطلاعات دانشجو را نمایش دهیم:

```
for key, value in student.items():
   print(f"{key}: {value}")
```

خروجی:

```
name: Ali
age: 23
city: Shiraz
```

۶-keys()

متد keys()تمامKey های Dictionary را برمی‌گرداند

```
student = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
print(student.keys())
```

خروجی:

```
dict_keys(['name', 'age', 'city'])
```

استفاده با for

```
for key in student.keys():
   print(key)
```

خروجی:

```
name
age
city
```

در بسیاری از موارد حتی می‌توانیم مستقیماً بنویسیم:

```
for key in student:
   print(key)
```

چون Loop مستقیم روی Dictionary نیزKey ها را پیمایش می‌کند

۷-values()

متد values()تمامValue های Dictionary را برمی‌گرداند.

```
student = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
print(student.values())
```

خروجی:

```
dict_values(['Ali', 23, 'Shiraz'])
```

استفاده با for

```
for value in student.values():
   print(value)
```

خروجی:

```
Ali
23
Shiraz
```

۸-pop()

متد pop()یک عنصر را بر اساس **Key**حذف می‌کند

```
student = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
```

```
student.pop("age")
print(student)
```

خروجی:

```
{'name': 'Ali', 'city': 'Shiraz'}
```

()popمقدار حذف‌شده را برمی‌گرداند

این نکته مهم است:

```
age = student.pop("age")
print(age)
```

خروجی:

```
23
```

بنابراین ()popهم حذف می‌کند و هم Value حذف‌شده را برمی‌گرداند

مقدار پیش‌فرض

اگر احتمال می‌دهیم Key وجود نداشته باشد: student.pop("phone", "Not Found")

اگر phoneوجود نداشته باشد، خطا ایجاد نمی‌شود و مقدار Not Found برگردانده می‌شود

popitem()-۹

متد ()popitemآخرین جفت Key-Value اضافه‌شده را حذف می‌کند

```
student = {
   "name": "Ali",
   "age": 23,
   "city": "Shiraz"
}
student.popitem()
print(student)
```

خروجی:

```
{'name': 'Ali', 'age': 23}
```

در Python 3.7 به بعد، Dictionaryترتیب ورود عناصر را حفظ می‌کند؛ بنابراین popitem()آخرین عنصر واردشده را حذف می‌کند

دریافت عنصر حذف‌شده

```
item = student.popitem()
print(item)
```

خروجی به شکل Tuple است:

```
('city', 'Shiraz')
```

پس:

```
key, value = student.popitem()
```

نیز امکان‌پذیر است

۱۰-setdefault()

این متد در ابتدا ممکن است کمی پیچیده به نظر برسد

setdefault()دو کار می‌تواند انجام دهد:

۱. اگر Key وجود داشته باشد، مقدار آن را برمی‌گرداند

۲. اگر Key وجود نداشته باشد، آن را ایجاد می‌کند

Keyوجود دارد

```
student = {
   "name": "Ali",
   "age": 23
}
x = student.setdefault("age", 30)
```

```
print(x)
print(student)
```

خروجی:

```
23
{'name': 'Ali', 'age': 23}
```

چون ageاز قبل وجود دارد، مقدار آن تغییر نمی‌کند

Keyوجود ندارد

```
student = {
  "name": "Ali",
  "age": 23
}
x = student.setdefault("city", "Shiraz")
print(x)
print(student)
```

خروجی:

```
Shiraz
{'name': 'Ali', 'age': 23, 'city': 'Shiraz'}
```

پس cityایجاد شد

مثال :

فرض کنید:

```
student = {
  "name": "Ali"
}
```

می‌خواهیم اگر ageوجود نداشت، مقدار پیش‌فرض 18ایجاد شود:

```
student.setdefault("age", 18)
```

```
print(student)
```

خروجی:

```
{'name': 'Ali', 'age': 18}
```

update()-۱۱

برای اضافه کردن یا تغییر چند Key-Value استفاده می‌شود

مثلاً:

```
student = {
   "name": "Ali",
   "age": 23
}
student.update({
   "age": 24,
   "city": "Shiraz"
})
print(student)
```

خروجی:

```
{'name': 'Ali', 'age': 24, 'city': 'Shiraz'}
```

در اینجا age وجود داشت، پس مقدارش تغییر کرد اما city وجود نداشت، پس اضافه شد

یکKey

```
student.update({
   "age": 25
})
```

چندKey

```
student.update({
   "age": 25,
   "city": "Baku",
   "country": "Azerbaijan"
})
```

۱۲- values() در محاسبات

متدهای Dictionary فقط برای نمایش نیستند

مثلاً:

```
grades = {
   "Python": 18,
   "Math": 17,
   "Database": 19
}
```

می‌توانیم مجموع نمرات را محاسبه کنیم:

```
total = sum(grades.values())
print(total)
```

خروجی:

```
54
```

میانگین:

```
average = sum(grades.values()) / len(grades)
print(average)
```

خروجی:

```
18.0
```

۱۳- ترکیب keys() و values()

```
grades = {
   "Python": 18,
   "Math": 17,
   "Database": 19
}
for subject in grades.keys():
   print(subject)
```

و:

```
for grade in grades.values():
   print(grade)
```

اما اگر هر دو را بخواهیم:

```
for subject, grade in grades.items():
   print(subject, grade)
```

۱۴- جدول متدهای Dictionary

Method	چه کاری انجام می‌دهد؟	مثال
clear()	همه را حذف می‌کند	d.clear()
copy()	کپی می‌سازد	d.copy()
fromkeys()	Dictionaryجدید می‌سازد	dict.fromkeys(keys, 0)
get()	مقدار Key را می‌دهد	d.get("name")
items()	Key + Value	d.items()
keys()	Keyها	d.keys()
pop()	حذف بر اساسKey	d.pop("age")
popitem()	حذف آخرینItem	d.popitem()
setdefault()	دریافت/ایجادKey	d.setdefault("city", "Shiraz")
update()	اضافه/تغییر	d.update({...})
values()	Valueها	d.values()

۱۶- مثال جامع

حالا یک برنامه نسبتاً واقعی بنویسیم:

```
student = {
    "name": "Ali",
    "age": 23,
    "major": "Computer Engineering",
    "gpa": 3.5
}
```

دریافت نام

```
print(student.get("name"))
```

نمایش Key ها

```
print(student.keys())
```

نمایش Value ها

```
print(student.values())
```

نمایش همه اطلاعات

```
for key, value in student.items():
    print(f"{key}: {value}")
```

تغییر سن

```
student.update({"age": 24})
```

اضافه کردن شهر

```
student.update({"city": "Shiraz"})
```

حذف GPA

```
student.pop("gpa")
```

ساخت Copy

```
backup = student.copy()
```

۱۷- پروژه کلاسی Student Dictionary :

یک Dictionary برای دانشجو ایجاد کنید:

```
student = {
   "name": "Ali",
   "age": 23,
   "major": "Computer Engineering",
   "gpa": 3.5
}
```

برنامه باید عملیات زیر را انجام دهد:

مرحله ۱ : نام دانشجو را با get()چاپ کنید

مرحله ۲ : تمامKey ها را با keys()چاپ کنید

مرحله ۳ : تمامValue ها را با values()چاپ کنید

مرحله ۴ : تمام اطلاعات را با items()چاپ کنید

مرحله ۵ : سن را به 24تغییر دهید

مرحله ۶ : شهر را به Dictionary اضافه کنید

مرحله ۷ : یک Backup با copy()ایجاد کنید

مرحله ۸ : gpaرا حذف کنید

مرحله ۹ : Dictionaryاصلی و Backup را چاپ کنید

تمرین ۱

Dictionaryزیر را ایجاد کنید:

```
car = {
   "brand": "Toyota",
```

```
    "model": "Corolla",
    "year": 2024,
    "color": "white"
}
```

با keys()تمامKey ها را چاپ کنید

تمرین ۲

با values()تمامValue ها را چاپ کنید

تمرین ۳

با items()تمام اطلاعات را به شکل زیر نمایش دهید:

```
brand: Toyota
model: Corolla
year: 2024
color: white
```

تمرین ۴

با get()مقدار modelرا دریافت کنید

تمرین ۵

با update()سال خودرو را به 2025تغییر دهید

تمرین ۶

با update()کشور را به شکل زیر اضافه کنید:

```
"country": "Japan"
```

تمرین ۷

با pop()مقدار colorرا حذف کنید

تمرین ۸

یک Copy از Dictionary ایجاد کنید و بررسی کنید که تغییر Dictionary اصلی روی Copy تأثیر نگذارد

تمرین ۹

با fromkeys()یک Dictionary برای درس‌های زیر ایجاد کنید:

```
subjects = [
   "Python",
   "Math",
   "Database",
   "English"
]
```

تمام نمرات را ابتدا 0قرار دهید

خروجی:

```
{
   'Python': 0,
   'Math': 0,
   'Database': 0,
   'English': 0
}
```

تمرین ۱۰

Dictionaryزیر را در نظر بگیرید:

```
grades = {
   "Python": 18,
   "Math": 17,
   "Database": 19,
   "English": 16
}
```

برنامه‌ای بنویسید که:

۱. مجموع نمرات را محاسبه کند

۲. میانگین را محاسبه کند

۳. بالاترین نمره را پیدا کند

۴. نام درسی که بالاترین نمره را دارد چاپ کند

۱۹- چالش نهایی

یک سیستم مدیریت دانشجو طراحی کنید:

```
student = {
   "name": "Ali",
   "age": 23,
   "major": "Computer Engineering",
   "grades": {
      "Python": 19,
      "Math": 17,
      "Database": 18
   }
}
```

برنامه باید بتواند:

نمایش نام دانشجو ، نمایش رشته ، نمایش تمام درس‌ها ، نمایش تمام نمرات ، محاسبه میانگین ، تغییر نمرهPython ، اضافه کردن درس جدید ، حذف یک درس ، ایجادBackup